Table III: Recommendation performance. The best performing method in each row is boldfaced, and the second best method in each row is underlined. Improvements over non-neural and neural approaches are shown in the last two columns respectively.

Dataset	Metric	(a) PopRec	(b) BPR	(c) FMC	(d) FPMC	(e) TransRec	(f) GRU4Rec	(g) GRU4Rec$^+$	(h) Caser	(i) SASRec	Improvement vs. (a)-(e)	Improvement vs. (f)-(h)
Beauty	Hit@10	0.4003	0.3775	0.3771	0.4310	<u>0.4607</u>	0.2125	0.3949	0.4264	**0.4854**	5.4%	13.8%
	NDCG@10	0.2277	0.2183	0.2477	0.2891	<u>0.3020</u>	0.1203	0.2556	0.2547	**0.3219**	6.6%	25.9%
Games	Hit@10	0.4724	0.4853	0.6358	0.6802	<u>0.6838</u>	0.2938	0.6599	0.5282	**0.7410**	8.5%	12.3%
	NDCG@10	0.2779	0.2875	0.4456	0.4680	0.4557	0.1837	<u>0.4759</u>	0.3214	**0.5360**	14.5%	12.6%
Steam	Hit@10	0.7172	0.7061	0.7731	0.7710	0.7624	0.4190	<u>0.8018</u>	0.7874	**0.8729**	13.2%	8.9%
	NDCG@10	0.4535	0.4436	0.5193	0.5011	0.4852	0.2691	<u>0.5595</u>	0.5381	**0.6306**	21.4%	12.7%
ML-1M	Hit@10	0.4329	0.5781	0.6986	0.7599	0.6413	0.5581	0.7501	<u>0.7886</u>	**0.8245**	8.5%	4.6%
	NDCG@10	0.2377	0.3287	0.4676	0.5176	0.3969	0.3381	0.5513	<u>0.5538</u>	**0.5905**	14.1%	6.6%

For fair comparison, we implement BPR, FMC, FPMC, and TransRec using *TemsorFlow* with the Adam [41] optimizer. For GRU4Rec, GRU4Rec$^+$, and Caser, we use code provided by the corresponding authors. For all methods except PopRec, we consider latent dimensions d from $\{10, 20, 30, 40, 50\}$. For BPR, FMC, FPMC, and TransRec, the ℓ_2 regularizer is chosen from $\{0.0001, 0.001, 0.01, 0.1, 1\}$. All other hyperparameters and initialization strategies are those suggested by the methods' authors. We tune hyper-parameters using the validation set, and terminate training if validation performance doesn't improve for 20 epochs.

C. implementation Details

For the architecture in the default version of SASRec, we use two self-attention blocks ($b = 2$), and use the learned positional embedding. Item embeddings in the embedding layer and prediction layer are shared. We implement SASRec with *TensorFlow*. The optimizer is the *Adam* optimizer [41], the learning rate is set to 0.001, and the batch size is 128. The dropout rate of turning off neurons is 0.2 for *MovieLens-1m* and 0.5 for the other three datasets due to their sparsity. The maximum sequence length n is set to 200 for *MovieLens-1m* and 50 for the other three datasets, i.e., roughly proportional to the mean number of actions per user. Performance of variants and different hyper-parameters is examined below.

All code and data shall be released at publication time.

D. Evaluation Metrics

We adopt two common Top-N metrics, Hit Rate@10 and NDCG@10, to evaluate recommendation performance [14], [19]. Hit@10 counts the fraction of times that the ground-truth next item is among the top 10 items, while NDCG@10 is a position-aware metric which assigns larger weights on higher positions. Note that since we only have one test item for each user, Hit@10 is equivalent to Recall@10, and is proportional to Precision@10.

To avoid heavy computation on all user-item pairs, we followed the strategy in [14], [48]. For each user u, we randomly sample 100 negative items, and rank these items with the ground-truth item. Based on the rankings of these 101 items, Hit@10 and NDCG@10 can be evaluated.

E. Recommendation Performance

Table III presents the recommendation performance of all methods on the four datasets (**RQ1**). By considering the second best methods across all datasets, a general pattern emerges with non-neural methods (i.e., (a)-(e)) performing better on sparse datasets and neural approaches (i.e., (f)-(h)) performing better on denser datasets. Presumably this owes to neural approaches having more parameters to capture high order transitions (i.e., they are expressive but easily overfit), whereas carefully designed but simpler models are more effective in high-sparsity settings.

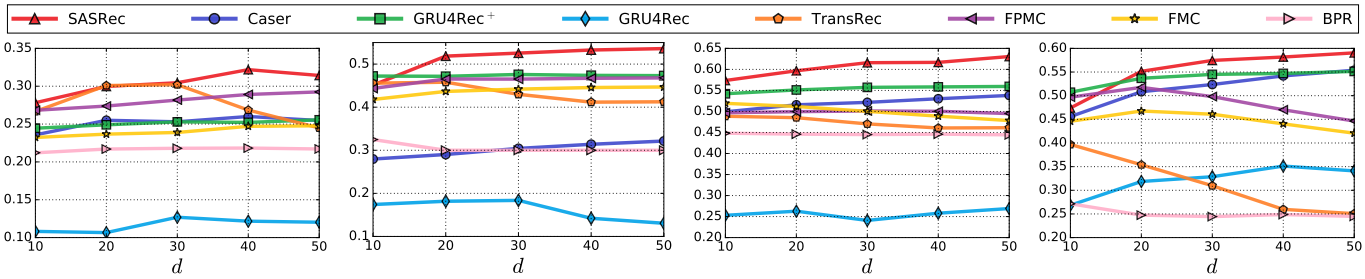
Our method SASRec outperforms all baselines on both sparse and dense datasets, and gains 6.9% Hit Rate and 9.6% NDCG improvements (on average) against the strongest baseline. One likely reason is that our model can adaptively attend items within different ranges on different datasets (e.g. only the previous item on sparse datasets and more on dense datasets). We further analyze this behavior in Section IV-H.

In Figure 2 we also analyze the effect of a key hyper-parameter, the latent dimensionality d, by showing NDCG@10 of all methods with d varying from 10 to 50. We see that our model typically benefits from larger numbers of latent dimensions. For all datasets, our model achieves satisfactory performance with $d \geq 40$.

F. Ablation Study

Since there are many components in our architecture, we analyze their impacts via an ablation study (**RQ2**). Table IV shows the performance of our default method and its **8** variants on all four dataset (with $d = 50$). We introduce the variants and analyze their effect respectively:

- (1) *Remove PE (Positional Embedding)*: Without the positional embedding $\mathbf{P}$, the attention weight on each item depends only on item embeddings. That is to say, the model makes recommendations based on users' past actions, but their order doesn't matter. This variant might be suitable for sparse datasets, where user sequences are typically short. This variant performs better then the default model on the sparsest dataset (*Beauty*), but worse on other denser datasets.



(a) *Beauty* (b) *Games* (c) *Steam* (d) *ML-1M*

Figure 2: Effect of the latent dimensionality d on ranking performance (NDCG@10).

Table IV: Ablation analysis (NDCG@10) on four datasets. Performance better than the default version is boldfaced. '↓' indicates a severe performance drop (more than 10%).

Architecture	*Beauty*	*Games*	*Steam*	*ML-1M*
(0) Default	0.3142	0.5360	0.6306	0.5905
(1) Remove PE	**0.3183**	0.5301	0.6036	0.5772
(2) Unshared IE	0.2437↓	0.4266↓	0.4472↓	0.4557↓
(3) Remove RC	0.2591↓	0.4303↓	0.5693	0.5535
(4) Remove Dropout	0.2436↓	0.4375↓	0.5959	0.5801
(5) 0 Block (b=0)	0.2620↓	0.4745↓	0.5588↓	0.4830↓
(6) 1 Block (b=1)	0.3066	**0.5408**	0.6202	0.5653
(7) 3 Blocks (b=3)	0.3078	0.5312	0.6275	**0.5931**
(8) Multi-Head	0.3080	0.5311	0.6272	0.5885

- (2) *Unshared IE (Item Embedding)*: We find that using two item embeddings consistently impairs the performance, presumably due to overfitting.
- (3) *Remove RC (Residual Connections)*: Without residual connections, we find that performance is significantly worse. Presumably this is because information in lower layers (e.g. the last item's embedding and the output of the first block) can not be easily propagated to the final layer, and this information is highly useful for making recommendations, especially on sparse datasets.
- (4) *Remove Dropout*: Our results show that dropout can effectively regularize the model to achieve better test performance, especially on sparse datasets. The results also imply the overfitting problem is less severe on dense datasets.
- (5)-(7) *Number of blocks*: Not surprisingly, results are inferior with zero blocks, since the model would only depend on the last item. The variant with one block performs reasonably well, though using two blocks (the default model) still boosts performance especially on dense datasets, meaning that the hierarchical self-attention structure is helpful to learn more complex item transitions. Using three blocks achieves similar performance to the default model.
- (8) *Multi-head*: The authors of Transformer [3] found that it is useful to use 'multi-head' attention, which applies attention in h subspaces (each a d/h-dimensional space). However, performance with two heads is consistently and slightly worse than single-head attention in our case. This might owe to the small d in our problem ($d = 512$ in Transformer), which is not suitable for decomposition into smaller subspaces.

G. Training Efficiency & Scalability

We evaluate two aspects of the training efficiency (**RQ3**) of our model: Training speed (time taken for one epoch of training) and convergence time (time taken to achieve satisfactory performance). We also examine the scalability of our model in terms of the maximum length n. All experiments are conducted with a single GTX-1080 Ti GPU.

Training Efficiency: Figure 3 demonstrates the efficiency of deep learning based methods with GPU acceleration. GRU4Rec is omitted due to its inferior performance. For fair comparison, there are two training options for Caser and GRU4Rec$^+$: using complete training data or just the most recent 200 actions (as in SASRec). For computing speed, SASRec only spends 1.7 seconds on model updates for one epoch, which is over 11 times faster than Caser (19.1s/epoch) and 18 times faster than GRU4Rec$^+$ (30.7s/epoch). We also see that SASRec converges to optimal performance within around 350 seconds on *ML-1M* while other models require much longer. We also find that using full data leads to better performance for Caser and GRU4Rec$^+$.

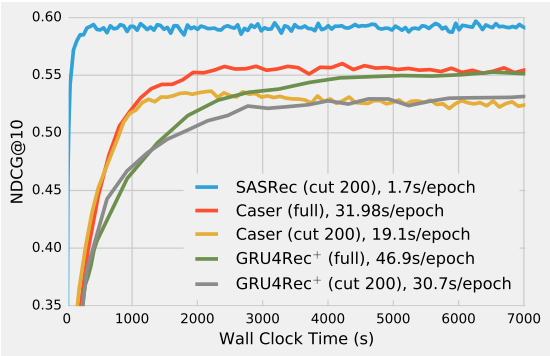
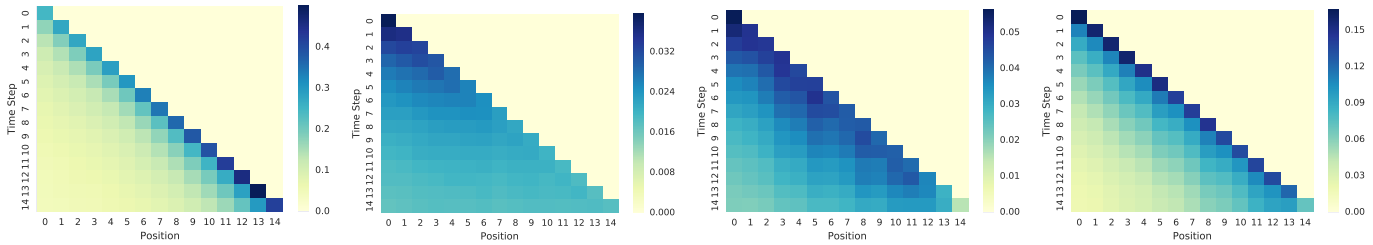


Figure 3: Training efficiency on *ML-1M*. SASRec is an order of magnitude faster than CNN/RNN-based recommendation methods in terms of training time per epoch and in total.



(a) *Beauty*, Layer 1 (b) *ML-1M*, Layer 1, w/o PE (c) *ML-1M*, Layer 1 (d) *ML-1M*, Layer 2

Figure 4: Visualizations of average attention weights on positions at different time steps. For comparison, the heatmap of a first-order Markov chain based model would be a diagonal matrix.

Scalability: As with standard MF methods, SASRec scales linearly with the total number of users, items and actions. A potential scalability concern is the maximum length n, however the computation can be effectively parallelized with GPUs. Here we measure the training time and performance of SASRec with different n, empirically study its scalability, and analyze whether it can handle sequential recommendation in most cases. Table V shows the performance and efficiency of SASRec with various sequence lengths. Performance is better with larger n, up to around $n = 500$ at which point performance saturates (possibly because 99.8% of actions have been covered). However, even with $n = 600$, the model can be trained in 2,000 seconds, which is still faster than Caser and GRU4Rec$^+$. Hence, our model can easily scale to user sequences up to a few hundred actions, which is suitable for typical review and purchase datasets. We plan to investigate approaches (discussed in Section III-F) for handling very long sequences in the future.

Table V: Scalability: performance and training time with different maximum length n on *ML-1M*.

n	10	50	100	200	300	400	500	600
Time(s)	75	101	157	341	613	965	1406	1895
NDCG@10	0.480	0.557	0.571	0.587	0.593	0.594	0.596	0.595

H. Visualizing Attention Weights

Recall that at time step t, the self-attention mechanism in our model adaptively assigns weights on the first t items depending on their position embeddings and item embeddings. To answer **RQ4**, we examine all training sequences and seek to reveal meaningful patterns by showing the average attention weights on positions as well as items.

Attention on Positions: Figure 4 shows four heatmaps of average attention weights on the last 15 positions at the last 15 time steps. Note that when we calculate the average weight, the denominator is the number of *valid* weights, so as to avoid the influence of padding items in short sequences.

We consider a few comparisons among the heatmaps:

- *(a) vs. (c)*: This comparison indicates that the model tends to attend on more recent items on the sparse dataset *Beauty*, and less recent items for the dense dataset *ML-1M*. This is the key factor that allows our model to adaptively handle both sparse and dense datasets, whereas existing methods tend to focus on one end of the spectrum.
- *(b) vs. (c)*: This comparison shows the effect of using positional embeddings (PE). Without them attention weights are essentially uniformly distributed over previous items, while the default model (c) is more sensitive in position as it is inclined to attend on recent items.
- *(c) vs. (d)*: Since our model is hierarchical, this shows how attention varies across different blocks. Apparently, attention in high layers tends to focus on more recent positions. Presumably this is because the first self-attention block already considers all previous items, and the second block does not need to consider far away positions.

Overall, the visualizations show that the behavior of our self-attention mechanism is *adaptive*, *position-aware*, and *hierarchical*.

Attention Between Items: Showing attention weights between a few cherry-picked items might not be statistically meaningful. To perform a broader comparison, using *MovieLens-1M*, where each movie has several categories, we randomly select two disjoint sets where each set contains 200 movies from 4 categories: *Science Fiction (Sci-Fi)*, *Romance*, *Animation*, and *Horror*. The first set is used for the query and the second set as the key. Figure 5 shows a heatmap of average attention weights between the two sets. We can see the heatmap is approximately a block diagonal matrix, meaning that the attention mechanism can identify similar items (e.g. items sharing a common category) and tends to assign larger weights between them (without being aware of categories in advance).

V. Conclusion

In this work, we proposed a novel self-attention based sequential model *SASRec* for next item recommendation. SASRec models the entire user sequence (without any recurrent or convolutional operations), and adaptively considers consumed items for prediction. Extensive empirical results on both sparse and dense datasets show that our model outperforms state-of-the-art baselines, and is an order of magnitude faster than CNN/RNN based approaches. In the future, we plan to extend the model by incorporating rich context information (e.g. dwell time, action types, locations, devices, etc.), and to investigate approaches to handle very long sequences (e.g. clicks).